

Level 10 → Level 11

base64로 암호화된 파일을 해독하기 위해 `base64 -d data.txt` 명령어를 사용해 비밀번호를 획득했다!

Level 11 → Level 12

`data.txt`가 알파벳을 13칸씩 밀어서 암호화했으므로 `tr` 명령어를 사용해 원래 알파벳으로 복원해준다. `cat data.txt | tr 'A-Za-z' 'N-ZA-Mn-za-m'` 명령어를 입력해 비밀번호를 획득했다!

Level 12 → Level 13

```
bandit12@bandit:~$ ls
data.txt
bandit12@bandit:~$ mktemp -d
/tmp/tmp.jwQc48h7Kj
bandit12@bandit:~$ cd /tmp/tmp.jwQc48h7Kj
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ cp ~/data.txt
cp: missing destination file operand after '/home/bandit12/data.txt'
Try 'cp --help' for more information.
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ cp ~/data.txt .
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ ls
data.txt
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ file data
data: cannot open 'data' (No such file or directory)
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ xxd -r data.txt > data
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ ls
data data.txt
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ file data
data: gzip compressed data, was "data2.bin", last modified: Wed Jun 24 14:58:46 2026, max compression, from Unix, original size modulo 2^32 580
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ mv data data.gz
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ gzip -d data.gz
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ ls
data data.txt
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ file data
data: bzip2 compressed data, block size = 900k
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ mv data data.bz2
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ bzip -d data.bz2
Command 'bzip' not found, but there are 22 similar ones.
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ bzip2 -d data.bz2
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ file data
data: gzip compressed data, was "data4.bin", last modified: Wed Jun 24 14:58:46 2026, max compression, from Unix, original size modulo 2^32 20480
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ mv data data.gz
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ gzip -d data.gz
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ file data
data: POSIX tar archive (GNU)
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ mv data data.tar
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ tar -xvf data.tar
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ file data
data: cannot open 'data' (No such file or directory)
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ ls
data.tar data.txt data5.bin
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ file data5.bin
data5.bin: POSIX tar archive (GNU)
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ tar -xvf data5.bin
ls
data.tar data.txt data5.bin data6.bin
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ file data6.bin
data6.bin: bzip2 compressed data, block size = 900k
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ mv data6.bin data6.bz2
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ bzip2 -d data6.bz2
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ ls
data.tar data.txt data5.bin data6
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ file data6
data6: POSIX tar archive (GNU)
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ tar -xvf data6
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ ls
data.tar data.txt data5.bin data6 data8.bin
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ file data8.bin
data8.bin: gzip compressed data, was "data9.bin", last modified: Wed Jun 24 14:58:46 2026, max compression, from Unix, original size modulo 2^32 49
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ mv data8.bin data8.gz
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ gzip -d data8.gz
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ ls
data.tar data.txt data5.bin data6 data8
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ file data8
data8: ASCII text
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ cat data8
The password is qQYQiH0BP8zR61qxYqX45quvihF2uzk
bandit12@bandit:/tmp/tmp.jwQc48h7Kj$ █
```

우선 처음 `mktemp -d` 로 작업용 폴더를 만들었다. 임시 폴더로 이동해서 `data.txt` 파일을 복제하였다. 문제에서 `data.txt`가 `hexdump`라고 했으므로 `xxd -r data.txt > data` 명령어로 `hexdump`내용을 원래 파일로 복구했다. 이후 `file data`로 파일의 종류를 확인한 다음 종류에 맞게 압축 해제를 계속 해주었다. `data8`의 종류가 아스키이므로 `cat data8`로 비밀번호를 획득했다! 지금까지 했던 레벨 중 가장 복잡하고 어려웠던 것 같다.

Level 13 → Level 14

`bandit13`으로 접속해 `ls`로 힌트와 개인키 파일이 있는 것을 확인하였다. 우선 `cat HINT`를 통해 힌트를 봤다. 현재 버전에서는 `local host` 접속 방식이 막혔다고 한다. 실제로 `ssh -i sshkey.private bandit14@localhost -p 2220`을 통해 접속하면 실패하는 것을 확인할 수 있었다. `exit`로 서버를 나온 후 `scp -P 2220`

`bandit13@bandit.labs.overthewire.org:sshkey.private ~/Desktop/sshkey.private` 명령어로 바탕화면에 개인키를 저장했다. 이후 `chmod 600 ~/Desktop/sshkey.private`로 권한을 제한했다. 이제 `ssh -i ~/Desktop/sshkey.private bandit14@bandit.labs.overthewire.org -p 2220` 명령어를 통해 비밀번호를 입력하지 않고 바로 `bandit14`에 접속했다. 이후 `cat /etc/bandit_pass/bandit14`를 입력해 비밀번호를 획득했다!

Level 14 → Level 15

현재 레벨의 비밀번호를 `localhost`의 30000번 포트에 보내면 다음 비밀번호를 알려준다고 했으므로 `nc localhost 30000`를 입력하고 비밀번호를 입력해서 다음 단계의 비밀번호를 획득했다!

Level 15 → Level 16

이번에는 SSH/TLS 암호화를 이용해 30001 포트로 비밀번호를 전송해야 한다. `openssl s_client -connect localhost:30001`로 암호화 한 후 비밀번호를 입력했더니 다음 단계의 비밀번호를 획득했다!

Level 16 → Level 17

문제대로 우선 `nmap -p 31000-32000 localhost` 를 입력해 열려있는 포트를 찾아냈다. 포트가 SSL/TLS인지 확인하기 위해 `nmap -sV -p 31000-32000 localhost` 를 입력했더니 꽤 오랜 시간 후 결과가 출력됐다. `-sV` 명령어는 포트에 대한 세부적인 정보를 알아내는 것이기 때문에 시간이 걸릴 수 있다고 한다. 그래서 열린 서버를 대상으로 `-sV`를 실행하기 위해 `nmap -sV -p 31046,31518,31691,31790,31960 localhost` 명령어를 실행했는데도 엄청 오래걸렸다. 그냥 `-sV` 명령어가 시간이 오래 걸리는 것 같다. 31790이 원하는 포트인것을 확인 후 `cat /etc/bandit_pass/bandit16 | openssl s_client -connect localhost:31790` 명령어로 개인키를 알아냈다. 이후 level 13 → level 14처럼 해서 bandit17서버에 접속 후 `cat` 명령어로 비밀번호를 획득했다!

Level 17 → Level 18

두 파일의 차이점을 이용해서 비밀번호를 알아내야 하므로 `diff` 명령어로 차이를 확인했다. 그 중 `password.new`에 있는 줄을 읽어 비밀번호를 획득했다! 주어진 비밀번호로 bandit18에 로그인했더니 Byebye! 라는 메시지와 함께 접속이 안됐다. 문제에서 다음 레벨의 힌트라고 했으니 다음 레벨로 넘어갔다.

Level 18 → Level 19

이번 레벨에서는 로그인할 때 `.bashrc`가 실행되어 자동으로 로그아웃이 된다고 한다. 그래서 접속과 동시에 명령어를 실행하기 위해 `ssh bandit18@bandit.labs.overthewire.org -p 2220 cat readme` 명령어를 실행 후 비밀번호를 입력했더니 다음 레벨의 비밀번호를 획득했다!

Level 19 → Level 20

우선 bandit19로 들어가 `ls -l` 를 통해 `bandit20-do`를 `setuid`를 통해 열어야 하는 것을 확인했다. `./bandit20-do`를 통해 열었더니 예시로 `./bandit20-do whoami`가 나와서 입력했더니 bandit20이 출력됐다. 즉 `./bandit20-do`로 명령어를 실행했을때 band20의 권한으로 실행되는 것을 확인했다. 이후 `./bandit20-do cat /etc/bandit_pass/bandit20`을 입력해 비밀번호를 획득했다!